Markdown Live Preview   Reset   Copy   Export PDF   ☐ Sync scroll   ☐ Dark mode

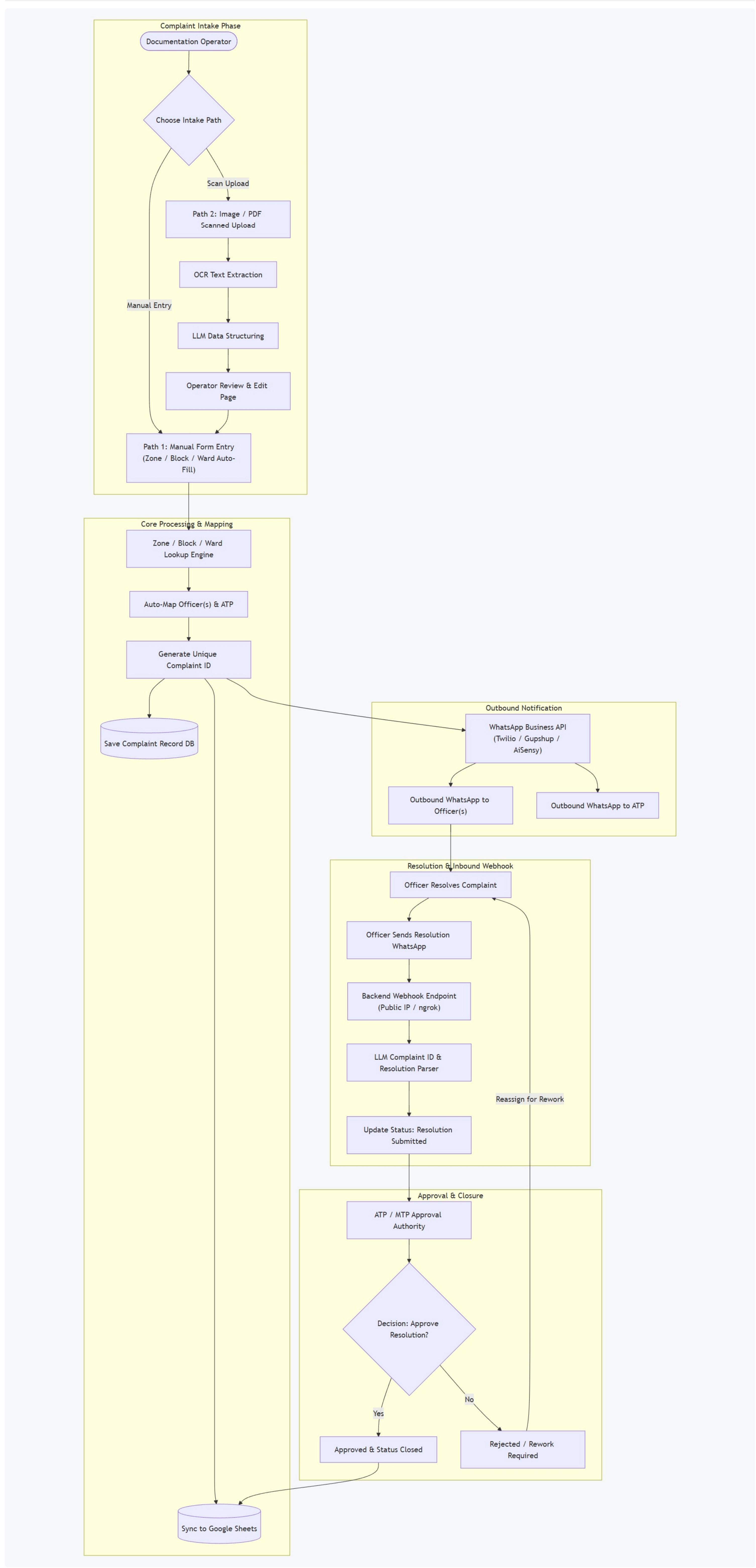
```
# MCL-BB — Full Plan

---

## Overview

MCL-BB is a complaint registration and management system for Ludhiana Municipal Corporation.
```

MCL-BB — Full Plan

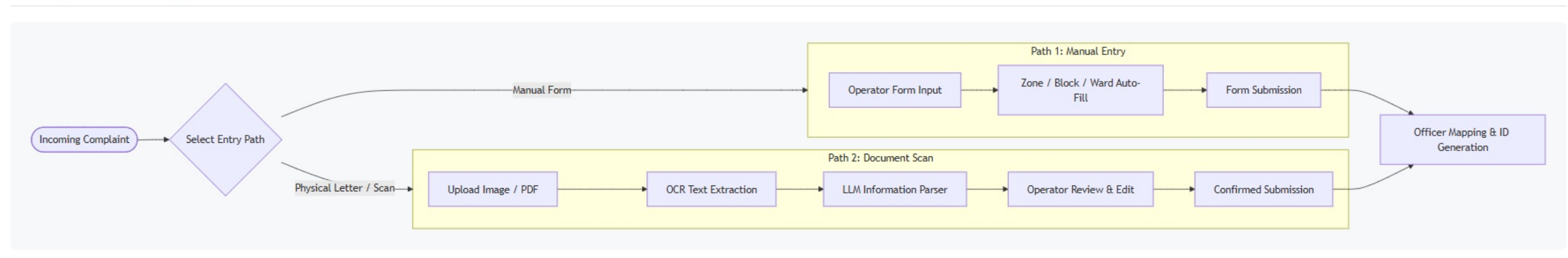
Overview

MCL-BB is a complaint registration and management system for Ludhiana Municipal Corporation. Complaints are received by citizens through calls, emails, or other channels and entered into the system by a documentation operator. There is no citizen-facing dashboard. The system handles location-based officer mapping, status tracking through a defined hierarchy, and resolution approval before closure.

End-to-End System Workflow


Complaint Intake Phase
Documentation Operator
Choose Intake Path
Scan Upload
Path 2: Image / PDF Scanned Upload
OCR Text Extraction
Manual Entry
LLM Data Structuring
Operator Review & Edit Page
Path 1: Manual Form Entry (Zone / Block / Ward Auto-Fill)
Core Processing & Mapping
Zone / Block / Ward Lookup Engine
Auto-Map Officer(s) & ATP
Generate Unique Complaint ID
Save Complaint Record DB
Outbound Notification
WhatsApp Business API (Twilio / Gupshup / AiSensy)
Outbound WhatsApp to Officer(s)
Outbound WhatsApp to ATP
Resolution & Inbound Webhook
Officer Resolves Complaint
Officer Sends Resolution WhatsApp
Backend Webhook Endpoint (Public IP / ngrok)
LLM Complaint ID & Resolution Parser
Update Status: Resolution Submitted
Reassign for Rework
Approval & Closure
ATP / MTP Approval Authority
Decision: Approve Resolution?
Yes
No
Approved & Status Closed
Rejected / Rework Required
Sync to Google Sheets

Complaint Entry — Two Paths


Incoming Complaint
Select Entry Path
Manual Form
Physical Letter / Scan
Path 1: Manual Entry
Operator Form Input
Zone / Block / Ward Auto-Fill
Form Submission
Path 2: Document Scan
Upload Image / PDF
OCR Text Extraction
LLM Information Parser
Operator Review & Edit
Confirmed Submission
Officer Mapping & ID Generation

Path 1: Manual Entry

The operator fills all fields in the complaint form. Every field is required; the form does not allow submission with missing data.

Fields:

- Citizen name
- Phone number
- Complaint title
- Complaint description
- Zone → Block → Ward
- Address
- Images / multimedia (upload)

Location Selection Logic:

Zone, Block, and Ward are interdependent but flexible in any direction of entry:

- If the operator selects **Zone** first → **Block** dropdown filters to blocks under that zone → **Ward** filters to wards under that block
- If the operator enters **Block** directly → **Zone** auto-fills → **Ward** filters accordingly
- If the operator enters **Ward** directly → **Zone** and **Block** both auto-fill

The data driving this logic lives in a JSON/DB lookup table. Nothing is hardcoded. The structure reflects Ludhiana's actual 4 zones and 13 blocks, with each ward mapped to its parent block and zone.

GIS Mapping (Optional):

The operator can optionally enter a pincode to drop a visual pin on a map. This uses Leaflet.js with OpenStreetMap — no paid API needed.

For MVP, GIS is a visual reference only. It does not drive officer mapping. The reason: pincode boundaries in India frequently overlap or cover multiple wards, and Ludhiana ward polygon/boundary data (GeoJSON) is not publicly available. Once MCL provides official boundary shapefiles, the map can be upgraded to confirm which ward a pincode falls in and use that for mapping. Until then, officer mapping is based entirely on the manually selected Ward.

On Submit:

1. System reads the selected Ward
2. Looks up the officer(s) assigned to that ward from the officer data
3. Looks up the relevant ATP
4. Generates a unique complaint ID
5. Saves complaint record to database and syncs to Google Sheets
6. Screen confirms: *"Complaint registered — assigned to [Officer Name], [Phone Number]"*
7. WhatsApp notifications sent to assigned officer and ATP

Path 2: Image / PDF Upload

The operator uploads a scanned image or PDF of the complaint (physical form, letter, etc.).

Processing flow:

1. File is passed through OCR to extract raw text
2. LLM processes the extracted text to identify and populate: ward, address, citizen name, phone, complaint description, and all other required fields
3. The operator is shown a preview/edit page with the auto-filled data
4. The operator reviews, corrects any errors, and confirms
5. On confirmation, the same submit flow as manual entry runs — officer mapping, complaint ID generation, DB + Sheets sync, notifications

Why a preview step: OCR accuracy on handwritten or low-quality scans is imperfect. The LLM extraction improves it significantly but cannot guarantee 100% accuracy. The operator's review step ensures no complaint goes in with wrong ward or citizen data.

WhatsApp Integration

WhatsApp is split into two separate problems with different complexity levels.

Sending Notifications (Outbound)

When a complaint is registered, a WhatsApp message is sent to the assigned officer and ATP containing the complaint ID, citizen name, ward, and complaint title.

How this works:

Use a WhatsApp Business API provider — Twilio, Gupshup, or AiSensy are suitable options for India. These providers handle Meta's API layer and offer simpler REST APIs to call from the backend.

The backend makes a POST request to the provider's API on complaint submission. The message content uses a pre-approved template (Meta requires all business-initiated WhatsApp messages to use approved templates).

Problems to solve before this works:

- A WhatsApp Business Account must be registered and verified with Meta
- Message templates must be submitted to Meta for approval (typically takes 1–3 days)
- The phone numbers of officers and ATPs must be stored and kept current in the system
- A provider account must be set up with sufficient balance/plan for the expected message volume

For MVP, this is the only notification direction needed. It is achievable within 1–2 weeks once the Business Account is ready.

Receiving Officer Updates (Inbound)

After an officer resolves a complaint, they send a WhatsApp message containing the complaint ID and resolution details. The system must receive this message, extract the complaint ID and resolution text, and update the complaint record.

How this works:

The WhatsApp Business API supports webhooks — when a message is received on the business number, Meta sends a POST request to a configured webhook URL on the backend. The backend receives this, passes the message text to an LLM, which extracts the complaint ID and resolution summary, then updates the complaint status to "Resolution Submitted" in the database and Sheets.

Problems to solve:

- The webhook endpoint must be publicly accessible (not localhost). During development, a tunneling tool like ngrok can expose a local server. In production, the server must be deployed with a real domain/IP.
- Officers may send messages in mixed Hindi/English or with inconsistent formatting. The LLM prompt must be robust enough to handle this and still extract a valid complaint ID.
- If the complaint ID in the message is wrong or missing, the system cannot match it to a record. A fallback is needed — either the system replies asking for clarification, or a manual lookup option is available on the portal.
- The officer's phone number must be registered in the system so the incoming message can be attributed to the right person. Unrecognized numbers should be flagged.

This part requires more testing and iteration than outbound notifications. It should be built and tested in a sandboxed environment before going live.

Officer Mapping

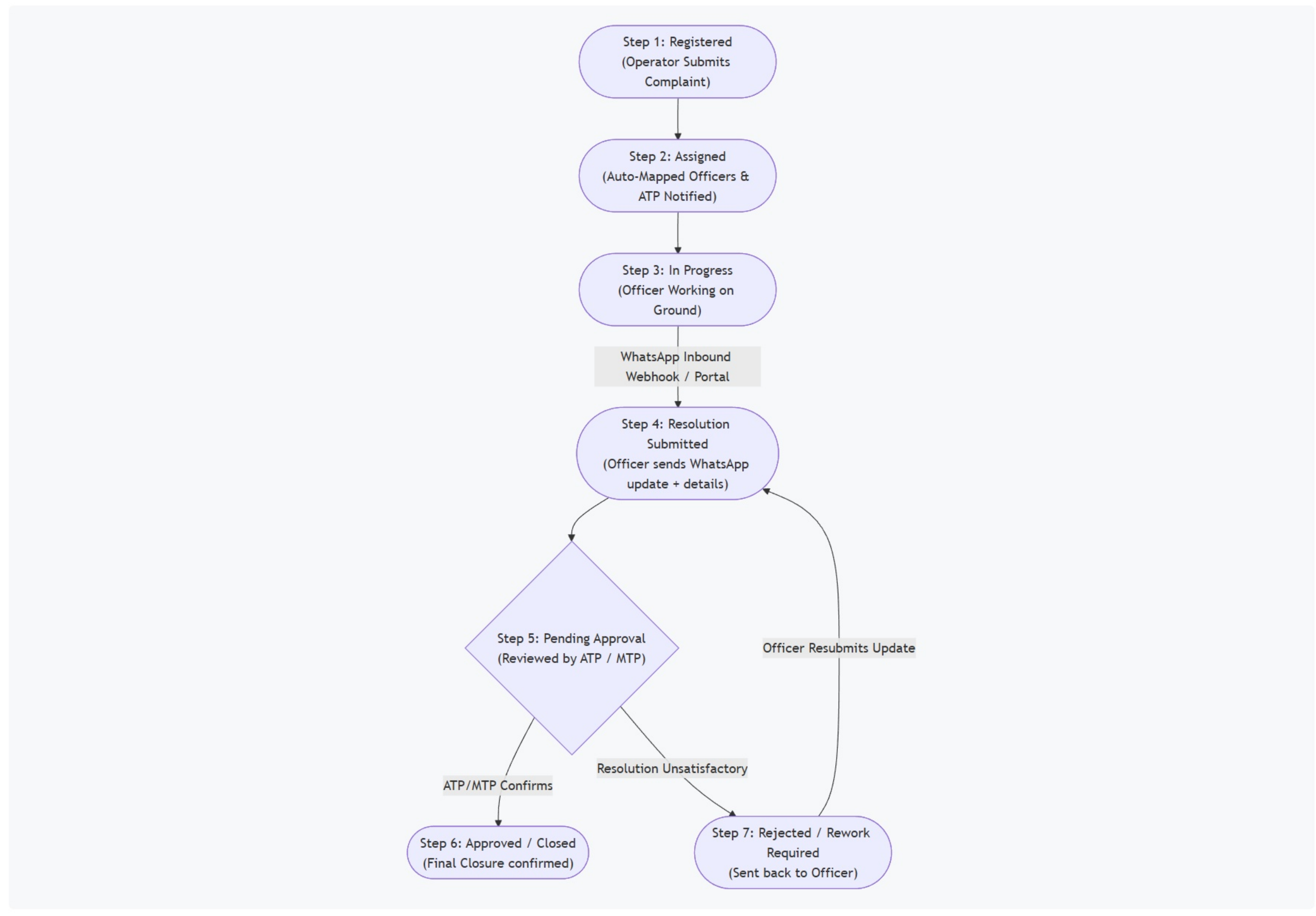
Officer data supports many-to-many assignment — one officer can cover multiple blocks and wards.

Officer record structure:

```
{
  "officerId": "OFF-001",
  "name": "Officer Name",
  "designation": "Inspector",
  "department": "B&R",
  "zone": "Zone-A",
  "blocks": ["Block-1", "Block-4"],
  "wards": ["Ward-2", "Ward-3", "Ward-11"]
}
```

When a complaint is submitted for a ward, the system queries all officers whose `wards` array includes that ward. If multiple officers are found, all are assigned and notified. ATP is mapped separately based on zone/block.

Complaint Status Workflow


Step 1: Registered (Operator Submits Complaint)
Step 2: Assigned (Auto-Mapped Officers & ATP Notified)
Step 3: In Progress (Officer Working on Ground)
WhatsApp Inbound Webhook / Portal
Step 4: Resolution Submitted (Officer sends WhatsApp update + details)
Step 5: Pending Approval (Reviewed by ATP / MTP)
Officer Resubmits Update
ATP/MTP Confirms
Resolution Unsatisfactory
Step 6: Approved / Closed (Final Closure confirmed)
Step 7: Rejected / Rework Required (Sent back to Officer)

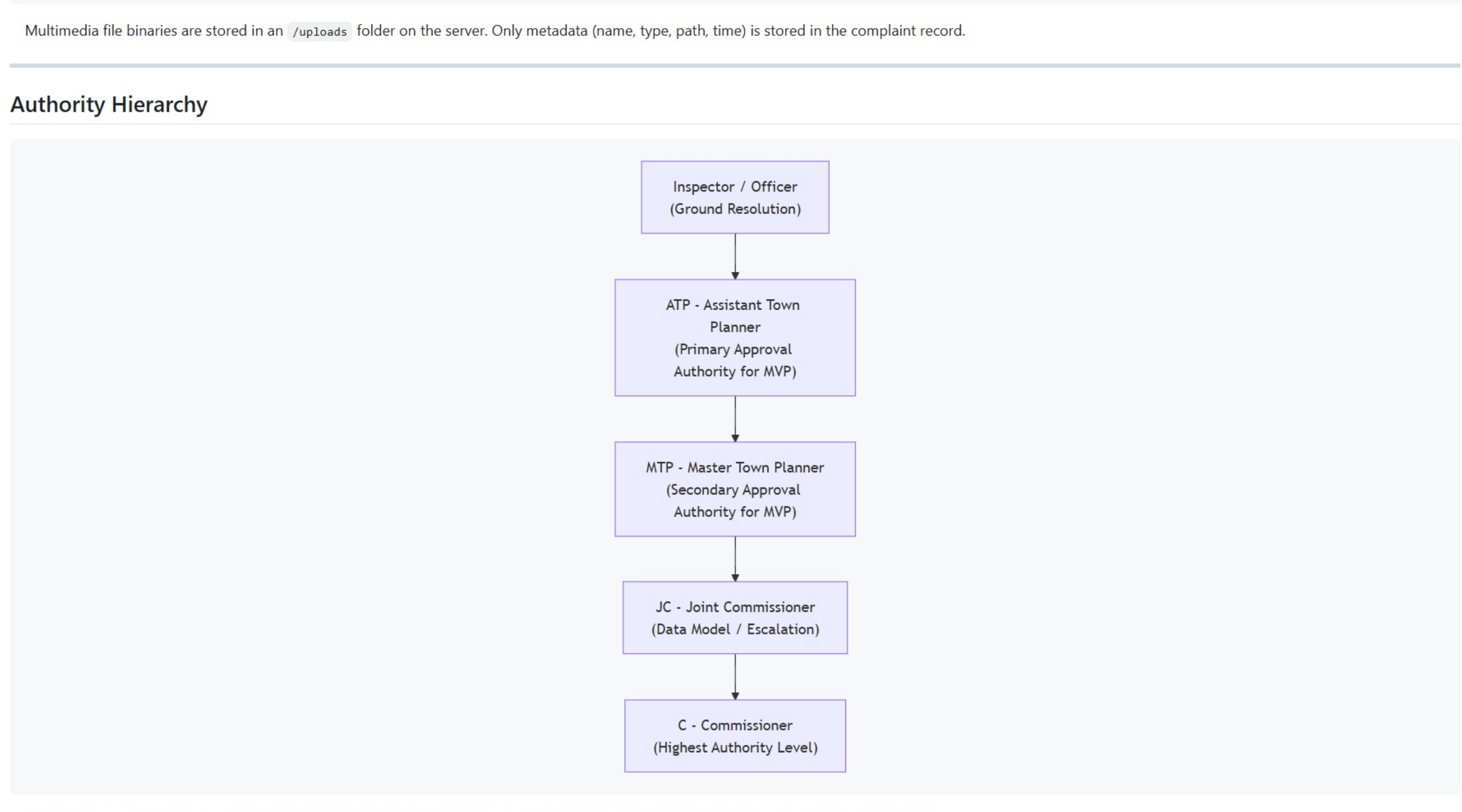
Resolution and closure are always separate states. An officer cannot close their own complaint.

Complaint Data Structure

```
{
  "complaintId": "CMP-2026-00842",
  "citizen": {
    "name": "Citizen Name",
    "phone": "+919876543210"
  },
  "complaint": {
    "title": "Road Repair Required",
    "description": "Potholes near main market area",
    "address": "Main Market Road, Ward 12"
  },
  "location": {
    "zone": "Zone-A",
    "block": "Block-2",
    "ward": "Ward-12",
    "gisPin": { "lat": 30.901, "lng": 75.857 }
  },
  "multimedia": [
    {
      "fileName": "pothole_photo.jpeg",
      "fileType": "image/jpeg",
      "filePath": "/uploads/2026/09/cmp-842-1.jpg",
      "uploadedAt": "2026-09-02T14:30:00Z"
    }
  ],
  "assignment": {
    "officers": ["OFF-001", "OFF-004"],
    "atp": "ATP-002"
  },
  "workflow": {
    "currentStatus": "In Progress",
    "resolution": null,
    "approvalStatus": "Pending",
    "approvalAuthority": "ATP"
  },
  "timestamps": {
    "createdAt": "2026-09-02T14:30:00Z",
    "resolutionSubmittedAt": null,
    "closedAt": null
  }
}
```

Multimedia file binaries are stored in an `/uploads` folder on the server. Only metadata (name, type, path, time) is stored in the complaint record.

Authority Hierarchy


Inspector / Officer (Ground Resolution)
ATP - Assistant Town Planner (Primary Approval Authority for MVP)
MTP - Master Town Planner (Secondary Approval Authority for MVP)
JC - Joint Commissioner (Data Model / Escalation)
C - Commissioner (Highest Authority Level)

For MVP, the approval step involves ATP or MTP. Higher levels (JC, C) are part of the data model but not actively involved in the MVP workflow.

Analytics Dashboard

The portal includes an analytics view accessible based on designation and login credentials. Login is username + area + designation based.

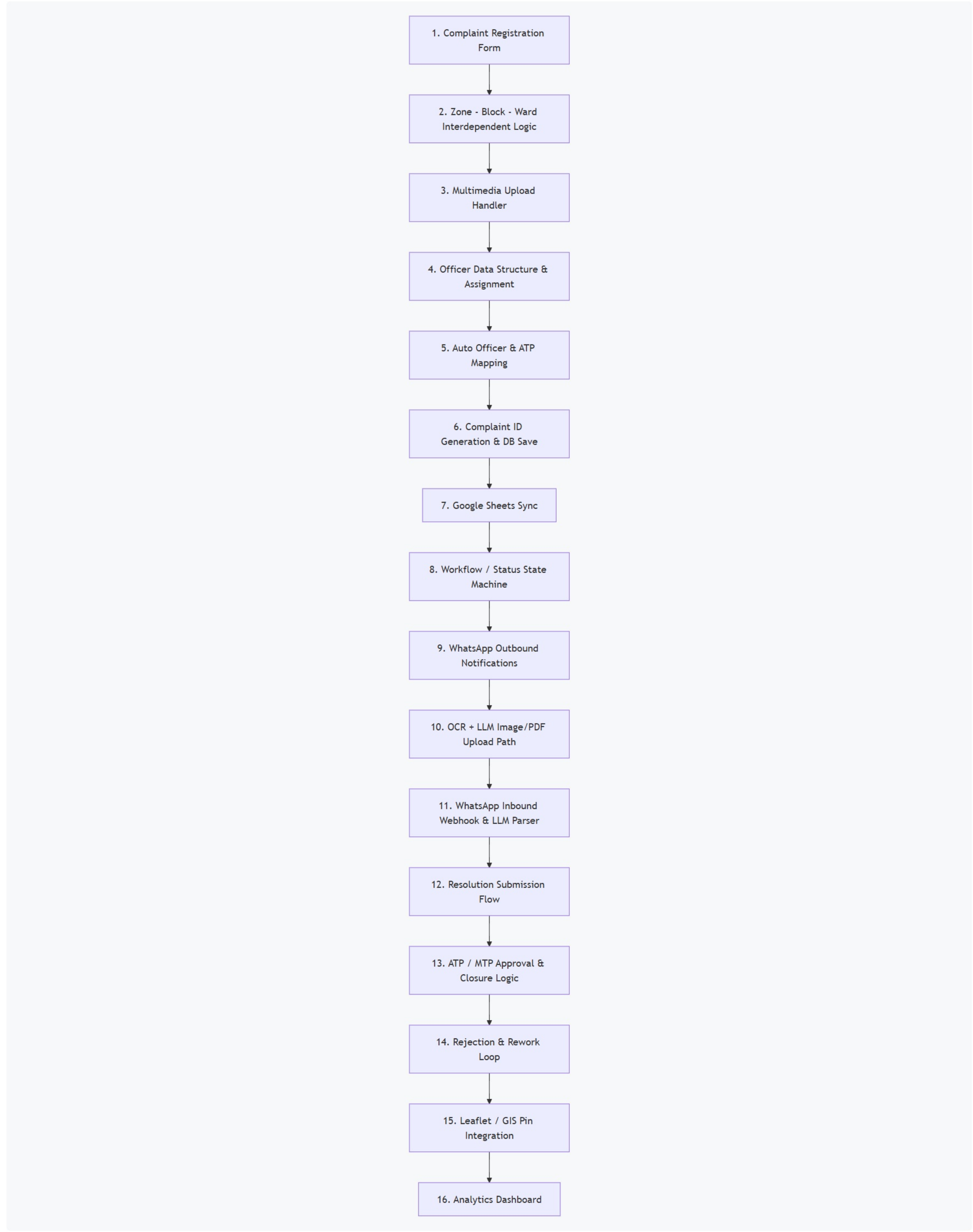
Dashboard shows:

- Complaint counts by status
- Filtering by zone, block, ward
- Top performers (officers with most resolved/approved complaints)

Google Sheets Sync

Every complaint registration and every status update syncs to a linked Google Sheet. This gives non-technical stakeholders a live view of all complaints without needing portal access. The sync uses the Google Sheets API with a service account.

Implementation Order


1. Complaint Registration Form
2. Zone - Block - Ward Interdependent Logic
3. Multimedia Upload Handler
4. Officer Data Structure & Assignment
5. Auto Officer & ATP Mapping
6. Complaint ID Generation & DB Save
7. Google Sheets Sync
8. Workflow / Status State Machine
9. WhatsApp Outbound Notifications
10. OCR + LLM Image/PDF Upload Path
11. WhatsApp Inbound Webhook & LLM Parser
12. Resolution Submission Flow
13. ATP / MTP Approval & Closure Logic
14. Rejection & Rework Loop
15. Leaflet / GIS Pin Integration
16. Analytics Dashboard

Key Rules (Non-Negotiable)

1. **No citizen dashboard or citizen login.**
2. Operator enters all complaints — received via call, email, or any channel.
3. All form fields are required.
4. Zone → Block → Ward are interdependent and auto-fill in any direction.
5. 4 zones, 13 blocks — data-driven, nothing hardcoded.
6. One officer can be assigned to multiple blocks and wards.
7. Officer and ATP mapping is automatic on submission.
8. Resolution submission does not close a complaint.
9. Closure requires explicit approval from ATP or MTP.
10. Resolution and closure are always separate states.
11. Multimedia binary never goes into the complaint record — metadata only.
12. GIS is optional and visual-only for MVP.
13. WhatsApp inbound requires a publicly accessible webhook — not available on localhost in production.
14. All status changes sync to Google Sheets.